



1+2

6

720 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

App Architecture and Career Pathways

د. روان قرعوني

محتوى مجاني غير مخصص للبيع التجاري

Programming Languages



RB Informatics; 27/10/2024

لغات البرمجة

أهلاً بكم يا أصدقاء في أولى محاضرات مادة لغات البرمجة، ننوّه على أن هذا العمل هو عمل طلابي بحث لا علاقة لدكتور المادة به، سترون خلاله معلومات ثرية ومهمة وملاحظات عملية تسبق المعلومات النظرية بأهميتها أحياناً. لندرس معاً ما هو نظري، ونستفيد مما هو عملي واقعي. ونرحّب بنوافذ ملاحظتنا تحت عنوان "نصائح عملية".

لغات البرمجة

في القسم العملي

40 علامة

في القسم العملي ستتناول جلساتنا معلومات عن التطبيقات كـ Back-end و Front-end بالتناوب بين الجلسات، التقييمات ستكون بـ flutter و laravel معاً.

في القسم النظري

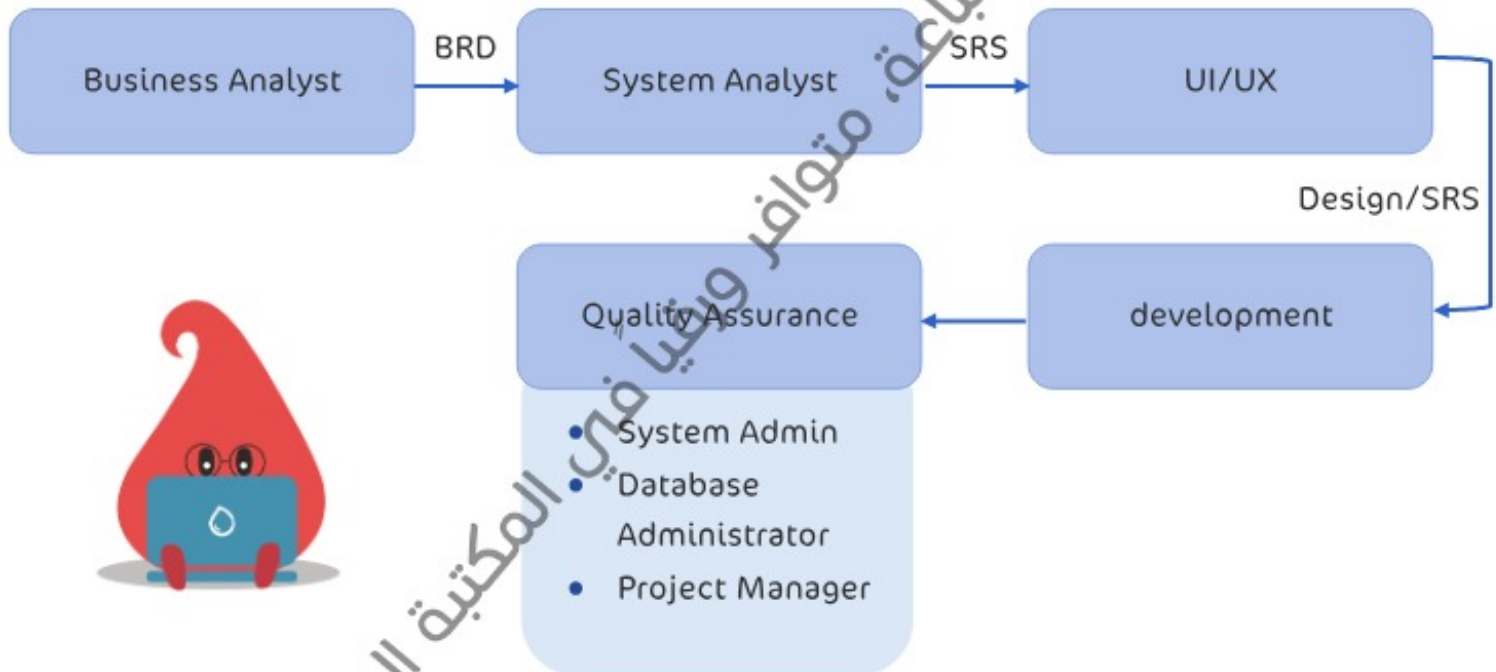
60 علامة

ستتنوع المعلومات بين نصائح عن سوق العمل وفرعنا الهندسي وأخرى عن تاريخ البرمجيات وما سنحتاج لتعلمه عن بنية المواقع والتطبيقات تمهيداً لما سينفذ في مادة "مشروع 1" لفصلنا القادم.

تعتبر هذه المادة أساس لباقي السنوات، تمهيداً للمشروع 1 والمشروع 2 ومشروع التخرج، لأنها عبارة عن تطبيقات ويب وموبايل مع الواجهات الرسومية و المنطق أي Back-end و Front-end، أي يمكن تنفيذهم بأكثر من لغة لكن في هذه المادة حصراً سيتم التطبيق بـ flutter للـ front و laravel للـ back، لتحقيق الهدف المرجو من تغيير منهج المادة وهو التقارب بين خريج كليتنا وسوق العمل.

نصائح عملية:

- لنا الحرية في اختيار framework التي سننفذ فيها تطبيقاتنا في "مشروع 1" أي يمكن أن نكمل ب-flutter أو نتعلم غيرها ك-react, native... بالنسبة لمجال front-end، أما بالنسبة لمجال back-end يمكننا العمل ب-laravel أو غيرها ك-.net, node js، لذلك ينصح بتعلم شيء جديد في كلا المجالين، لاكتساب خبرة أكبر ولأن كثرة المطورين ب-flutter و laravel في سوق العمل الذي أدى إلى قلة الفرص وصعوبة إيجاد عمل فيها:
- يجب الاطلاع على أساسيات القسم الآخر في حال كنا front-end أو back-end، لمعرفة كل قسم ماذا يقدم للآخر خلال العمل، ولتفادي الأخطاء وسوء الفهم، زيادة الخبرة وتحقيق نتائج أفضل وأكثر احترافية.
- لا يمكن الاعتماد على جلسات القسم العملي للمادة لتنفيذ المشروع، فالمادة تعتمد على التعلم الذاتي.

الوظائف التي يمكن أن يشغلها مهندس المعلوماتية:**مقدمة:**

هناك تصور خاطئ أن خريجي كليتنا من مهندسين، يمكنهم فقط أن يكونوا مبرمجين ومطورين، قد يكون الجزء الأكبر منهم مبرمجين فعلاً، لكن هذا ليس المكان الوحيد الذي يمكن أن يُشغل وستحدث الآن عن اختصاصات تتطلب أن يشغلها مهندس معلوماتي:

عند بدء الشركة بأي مشروع برمجي، مطعم أو صيدلية أو...، يلتقي العميل أولاً مع:

1. Business Analyst

محل الأعمال يلتقي بالعميل ويجمع المتطلبات، يجب أن يكون ذو خبرة برمجية سابقة، لا تقتصر مهمته فقط على جمع المتطلبات، يجب أن يمتلك المهارات لجمعها بشكل صحيح بحيث تكون قابلة للتنفيذ أي متطلبات منطقية إضافة لكونها

متطلبات متكاملة يقوم بالتأكد من انعكاساتها ويقترح ما هو أفضل للقيام به، لأنه في سوق العمل، لا يمكن للعميل أن يحصي ويطلب بالضبط ما يمكن أن يتواجد بالتطبيق المطلوب.

تتراوح المقابلات بين BA والعميل من 4 إلى 10 مقابلات حسب حجم المشروع المطلوب.

يوسط المهندس BA المعلومات في ملف هو: Business Requirements Document (BRD)

توصيف متطلبات العمل _ ولهذا الملف قالب معين ستتعرف عليه في مادة "هندسة البرمجيات" لاحقاً. ثم يصل ملف BRD إلى:

2. System Analyst

محلل النظام _ يوصف المتطلبات بحالات استخدام (use cases)، مثال: إحدى المتطلبات هي وجود (login).
الـ use cases هي:

- في حال إدخال اسم المستخدم بشكل خاطئ، تظهر رسالة على الشاشة
- في حال إدخال كلمة السر بشكل خاطئ، تضيء النافذة بلون معين
- في حال إدخال كلاهما بشكل صحيح يتم تسجيل الدخول وهكذا..

كل مطلب يتم مناقشته وتفصيل حالاته بملف: Software Requirements Specification (SRS)،

يحتوي: Use case Diagram

Use case specification

Login
Start
Back

ينشئ محلل النظم ملفاً رسمياً يحوي فقط ما هو مطلوب تصميمه

ثم يصل الملفان السابقان إلى:

3. UI/UX team

يخرج فريق التصميم من الملفين السابقين (التوصيف، الملف الرسومي) بشاشات نهائية مستخدمين أدوات مثل Figma و Xd وغيرها من برامج التصميم يكون المخرج النهائي عبارة عن شاشات رسومية بالألوان وأبعاد محددة مع الشعار المحدد للعميل، بشكل جميل ومدرس من حيث التوضع للأزرار والنوافذ وأنواع الخطوط وأشكالها والألوان المستخدمة.

يمكن لهذا الموقع أن يشغل من قبل مهندس معلوماتي قد درس في مجال UI/UX أو من قبل مصممين ليسوا بمهندسين لكن يفضل أن يكون المصمم مهندساً لما يعلمه من أساسيات برمجية قابلة للتنفيذ وخبرة في فهم المبرمج.

الفرق بين UI و UX:

User Experience	User Interface
• تجربة المستخدم • مسؤول عن جعل الشاشة سهلة الاستخدام ومريحة للمستخدم • يتحكم بتموضع الأزرار والنوافذ، حجمها وثباتها واختفاؤها أو ظهورها	• واجهة المستخدم • مسؤول عن ظهور الشاشة بشكل جميل • يتحكم باللون والخط بحجمه وأشكاله والصور والرموز الموضوعة

يمكن أن يكون الفريق مكون من 1 إلى 10 أشخاص مسؤولاً عن كلا القسمين UI/UX أو أن يكون القسمان منفصلان وهذا يعتمد على حجم العمل في الشركة.
ينتقل ملفا SRS والتصميم النهائي للشاشات او الواجهات إلى قسم الـ:

4. Development

- يكون القسم الأكبر من الشركة مكوناً من المطورين والمبرمجين بنسبة (70_80%)، لأنه قسم يتفرع للكثير من الاختصاصات باختلاف اللغات المستخدمة ونوع الكائن البرمجي الذي يتم إنشاؤه.
- يُعنى قسم الـ Front-end في فهم التوصيف وكتابة كود الشاشة المرسومة، بحيث يفيد فهم التوصيف في اختبار فعالية الشاشة المبرمجة.
- يُعنى قسم الـ Back-end بالاطلاع على التصميم المرسوم للواجهات وبرمجة المنطق (logic) الذي يعطي الواجهات فعاليتها، والمأخوذ من ملف التوصيف (SRS).
- لا يستلم العميل مشروعه عند انتهاء المبرمجين من كلا القسمين وربطهما، بل يمر أولاً على فريق:

5. Quality Assurance

- يستلم فريق ضمان الجودة ملف التوصيف والتصميم ويجرب حالات اختبار test cases لكل النظام المنشأ. بحيث يجرب كل الحالات والتباديل الممكنة؛ لإيجاد أي خطأ قد وقع في عمليات الإنشاء السابقة، كي يضمن وصول المشروع للتعليق صحيحاً دون أخطاء.
- يجري الفريق العديد من عمليات الاختبار فلا يكتفي فقط بتجريب التطبيق، يقوم بمقارنة التطبيق النهائي مع ملف التصميم، وإذا ما كانت الواجهات متوافقة مع مختلف أحجام الشاشات، واختبار أمني في حال هو قابل للاختراق، وهل هو قابل للضغط.
- يوجد طريقة أوتوماتيكية للاختبار بحيث لا يضطر الفريق لإعادة المراحل مراراً أو إدخال أعداد كبيرة من المستخدمين، بل تقوم الأدوات المبرمجة بتجريب كل الاختبارات.
- عند انتهاء العملية يتم رفع المشروع على السيرفرات (Deploy) ويوجد **System Admin** وهو مهندس معلوماتي يشرف على السيرفرات ويأخذ نسخ مرجعية، ويقوم أيضاً **Database Administrator** بالتأكد من أن جميع البيانات محفوظة دون خلل، يقوم بنسخ البيانات من أجل الرجوع السريع لها في حال حدوث أي خطأ.
- ويدير المشروع ككل **Project Manager** ويتابع جميع الفرق ويكون مبرمج سابقاً وعلى علم بالأساسيات العملية للتطبيقات، ويمكن أن يدير أكثر من مشروع في نفس الوقت.



Your greatest fear is not that
you will fall, your greatest fear
is that you will live a full life
and never fly

كيف وصلت تطبيقات الويب والموبايل إلى شكلها الحالي؟

في بداية الستينيات كانت التطبيقات عبارة عن شاشة سوداء (command line) ثم تطورت إلى واجهة زر وجدول مع إدخال المطلوب، عبارة عن تطبيق واحد دون تواصل مع السيرفرات أو العميل، فقط يحوي جزء (logic)، أي تبنى الأدوات وتخزن البيانات في نفس المكان.

مع وجود سلبية وهي عدم قدرة الأجهزة على التواصل مع بعضها، ثم استحدث في السبعينات مفهوم الشبكات ثم عام 1989 تم اختراع مفهوم (word wide web) من الحاجة إلى التبادل المعلوماتي، ومشاركة المعلومة مع الآخر، وهكذا استحدث التواصل على مستوى العالم ثم ظهر مفهوم "صفحات الويب"، التي تحوي المعلومات وتتواصل عن طريق الإنترنت، بحيث يكون هو صلة الوصل بين الصفحات التي تقدم المعلومات.

ولتمييز هذه الصفحات عن بعضها وجد الـ URL، وهو معرف الصفحة حيث لا يوجد صفحة لها URL مطابق لـ URL صفحة أخرى. (في الفقرة القادمة سنتحدث عن عمله مع IP و DNS).

وكما نعلم أن كل صفحة تقدم معلومة، قد تحتاج التنقل بين عدة صفحات لتكوين صورة ومعلومات متكاملة عن معلومة محددة، وهنا ظهر ما يسمى بـ Hyper Text يحاكي عقلية الإنسان عند التعامل مع معلومة متشعبة، بحيث يحتاج إلى تناول معلومات فرعية تصب في معنى المعلومة الأساسية وكذلك صفحات الويب استخدمت Hyper Texts Links كي تكون كل صفحة مترابطة مع الصفحات الأخرى، بحيث يتم التنقل بينها للحصول على المعلومة كاملة. وظهر أيضاً Hyper Media أي استخدام الصور والمقاطع المتحركة والأصوات في الصفحات.

وهنا تساؤل، كيف بنيت هذه الصفحات؟

بنيت بوساطة لغات توصف المعلومة وهي HTML (Hyper Text Markup Language) وهي مسؤولة عن تموضع الكائنات في الواجهة، أما CSS (Cascading Style Sheets) فهي لغة مسؤولة عن الألوان والأشكال للصور والنصوص، وأخيراً JS (Java Script) وهي اللغة المسؤولة عن قدرة المستخدم على التفاعل مع الكائنات ضمن الصفحة.

كيف أنشأت المواقع Websites؟

هي مجموعة من صفحات الويب web pages المترابطة والمتناسقة من حيث المحتوى والتصميم ومعمارية الموقع (أي يتم بناء الموقع بطريقة سهلة التنقل بين صفحاته)، ولأن الـ websites تكون static لا نستطيع التعديل على الـ Data الموجودة فيها ظهرت web application التي تتيح التفاعل والتعامل مع الداتا كالحذف والإضافة والتعديل. وأفادت الـ Database بسهولة التعامل مع البيانات حيث تضعها في جداول مترابطة.

بنى التطبيقات البرمجية Software Architecture

- 1 tier Architecture (single process)
- 2 tier Architecture (client/server) fat client
- N tier Architecture

1 tier Architecture:

مثل Desktop app و command line أو الألعاب الالكترونية التي لا تحتاج إلى اتصال بالانترنت أو أي تطبيق على الموبايل بدون server ويكون عبارة عن طبقة واحدة تحوي الواجهات والمنطق والبيانات فكان لها الكثير من السلبيات مثل انعدام التواصل وعدم القدرة على نقل البيانات وأنها محدودة القدرات تعتمد على امكانيات الهارد و الـ RAM والمعالج ثم تطورت إلى:

2 tier Architecture:

قسمت التطبيقات إلى (server/client) ولكل قسم تعديلاته الخاصة لكن احتوى على سلبية وهي أن client احتوى على جزء من المنطق ففي حال حصول أي خطأ وجب التعديل على كل الأجهزة فظهر مفهوم هو: fat client وهو تركز المنطقي في قسم client نتيجة لكثرة التعديلات والتطويرات وأخيراً:

N tier Architecture:

قسم التطبيق إلى ثلاث طبقات:



GUI: يحتوي على واجهات التطبيق والموبايل ولا يحتوي على أي منطق إنما فقط على الشاشات الرسومية.
 Logic: وهذا القسم مسؤول عن معالجة الـ APIs ولا يحتوي على أي داتا.
 Data: في هذا القسم تتواجد الـ Database ولا تتم به أي عملية معالجة فقط ارسال واستقبال بيانات.



THE END